

6. PROJECT TESTING PHASE

6.1 Testing Strategy and Test Levels

Testing verifies that the application meets its requirements and handles errors predictably. Unit tests cover calculation functions, validation rules and data conversions. Integration tests check that interface actions, application logic and database operations work together. System tests exercise complete workflows, while user acceptance testing gathers feedback from representative users such as scorers or coaches.

Test cases should include normal, boundary and invalid inputs. Normal cases represent typical scorecards. Boundary cases include zero balls faced, zero dismissals, no bowling record and a player with no performance in a selected period. Invalid cases include negative runs, impossible dates, duplicate identifiers and references to missing players.

Expected behavior should be defined before testing. A test log can record test ID, requirement, input, expected result, actual result, status and defect reference. Defects should be prioritized by their effect on data correctness and core workflows. A failed test should be repeated after correction, along with related regression tests.

Testing should be continuous rather than postponed until the final week. Early tests can reveal unclear requirements or data-model problems before they become expensive to correct. Evidence such as sample scorecards, screenshots and calculation tables can support the final project report.

6.2 Functional and Calculation Testing

Functional testing checks whether features behave as specified. Player creation should save valid data and reject missing required fields. Match creation should enforce valid team relationships. Performance entry should reject negative values and identify incomplete records. Search should return matching results and handle no-result cases. Export should contain the same records represented by the selected report.

Calculation testing is critical because an attractive dashboard can still be misleading if formulas are incorrect. Prepare small scorecards and calculate expected values independently. A batter scoring 30 runs from 20 balls has a strike rate of 150.

A batter scoring 60 runs with two dismissals has a batting average of 30. A bowler conceding 24 runs in four overs has an economy rate of 6.

Overs notation must be tested separately. A bowler delivering 26 legal balls has bowled four overs and two balls, not 4.2 decimal overs. The calculation should use 26 legal balls. Tests should also cover zero dismissals, zero balls faced and zero legal balls, with the application displaying the documented unavailable representation rather than crashing.

The test dataset should include cases involving extras and not-out innings according to the selected scoring rules. When a source value is corrected, reports should reflect the change. Test results should be retained as regression cases so later development does not reintroduce known calculation errors.

6.3 Usability, Performance and Security Testing

Usability testing evaluates whether users understand the navigation, terminology and feedback. Ask a scorer to enter a sample match without assistance and observe where confusion occurs. Check whether required fields are obvious, validation messages are helpful and player history or reports are easy to locate. Feedback should lead to specific interface improvements.

Performance testing should use a dataset larger than the smallest demonstration example. Measure whether search, match retrieval and report generation remain responsive for the expected workload. If the system slows down, review database indexes, repeated queries and unnecessary recalculation. Document the test environment and dataset size rather than making unsupported claims about scalability.

Security checks should verify that restricted actions are unavailable to unauthorized users when authentication is implemented. Inputs should be validated, credentials protected and exports limited to necessary information. Backup and restore procedures should be tested. A local academic prototype may have limited controls, but known limitations should be documented before any real-world use.

Compatibility testing can check the application on the intended operating system and supported browser or runtime. Installation should be tested using the documented instructions. Error handling should be checked for database unavailability, missing dependencies and invalid user actions. These tests help establish whether the application is usable beyond the developer's own environment.

6.4 Defect Correction, Regression and Test Results

When a defect is identified, record how to reproduce it, the expected result, actual result and affected requirement. Correct the underlying cause rather than hiding the symptom. For example, an incorrect economy rate may arise from treating overs notation as a decimal. The correction should standardize internal storage as legal balls and update related display and tests.

Regression testing reruns previously passed tests after a change. This is important when editing database relationships, calculation functions or shared interface components. A change to a calculation should trigger tests for relevant player summaries, match reports and exports. A database migration should be tested with both new and existing records where applicable.

The final test summary should report the number of tests executed, passed, failed and deferred. Unresolved defects should be described with their impact and workaround if one exists. A feature should not be described as fully tested if it was only demonstrated once without defined expected results.

Test evidence may include screenshots, sample input files, expected-versus-actual tables and a defect log. The report should distinguish verified behavior from planned improvements. Transparent reporting makes the project easier to evaluate and gives future developers a useful maintenance starting point.